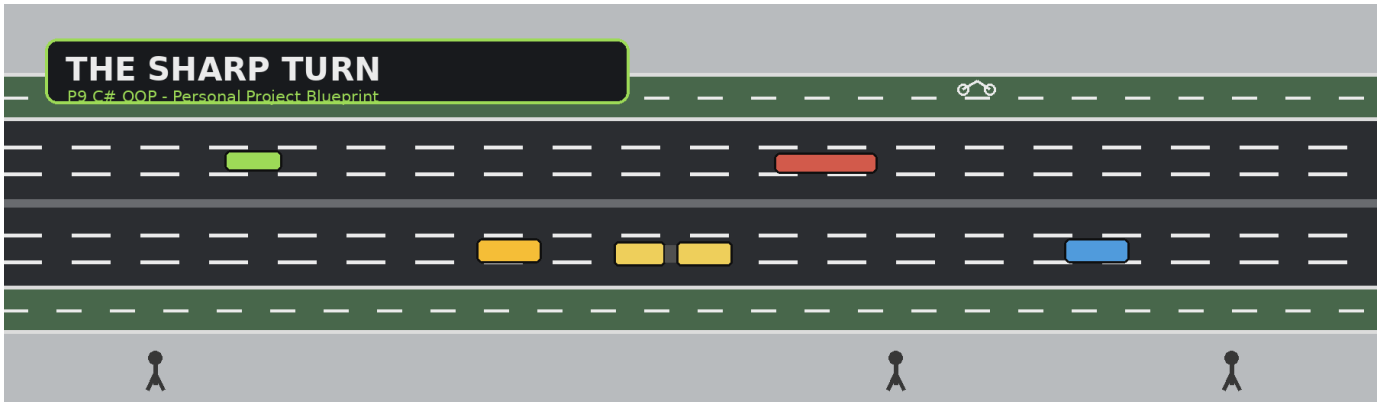

THE SHARP TURN

Complete Personal Project Rundown

Current design bible - class planning, movement rules, placement, manual control, audio, serialization, and build phases

CORE IDEA A top-down 2D C# WinForms traffic sandbox/editor. The assignment core is Add / Modify / Delete + polymorphism + graphical representation + serialization. The extra identity is hands-on manual control of cars, bikes, and pedestrians using one reusable movement philosophy.

Project style

Top-down GTA 1 / GTA 2 inspiration
Simple enough to explain line-by-line
No game engine or advanced libraries

Hard scope rule

Use only HIT Year-1 material
Intro CS + Advanced Programming + OOP
Standard WinForms/base components are fine

Current stage

Phase 1 behavior design: essentially locked
Today: finish Phases 2-4 only
Tomorrow: Phases 5-8

What makes it ours

Playable manual control
Context-sensitive lane/path rules
Simple WAV feedback via SoundPlayer

1 Assignment Fit & Scope

Everything below is designed to stay obviously inside the P9 brief rather than becoming a giant game project.

P9 requirement	How The Sharp Turn satisfies it
Custom hierarchy	TrafficObject -> RoadUser / SidewalkUser / BikePathUser, with concrete derived classes beneath them.
Polymorphism	The collection stores TrafficObject references while Draw / Move behavior is provided by the real derived type.
Collection object	TrafficObjectList, following the course-style SortedList approach.
Add / Modify / Delete	All entities are manually created, editable, selectable, and removable through the UI.
Event-driven GUI	WinForms button, mouse, keyboard, Paint, Timer, Save/Open events.
Graphical representation	Top-down road, bike path and pavement with visually distinct entity models.
Serialization	Real save files restore the current object system through FileStream + course-style serialization.

> **IMPORTANT** "Event-driven" means the WinForms program reacts to clicks, mouse movement, keys, Timer ticks, etc. It does NOT require random event simulation or engineering reports.

Explicitly scrapped

- No automatic/random entity spawning or spawn-rate system.
- No traffic-light optimization, throughput/wait-time experiments, or engineering reports.
- No 4-way intersection for the foundation map; the road is two opposing carriageways.
- No crash/wreck lifecycle, tow-truck dispatch, ambulance dispatch, injury logic, cleanup timer, or realistic physics.
- No complex pathfinding, automatic emergency routing, game engine, advanced audio engine, or multithreading.

2 Class Hierarchy

Three movement environments, one shared TrafficObject base.

```
TrafficObject
|
+-- RoadUser
|   +-- Car
|   +-- Motorcycle
|   +-- Bus
|   +-- EmergencyVehicle
|
+-- SidewalkUser
|   +-- Pedestrian
|
```

```
+-- BikePathUser
    +-- Bicycle
```

Class / group	Purpose
TrafficObject	Universal state and behavior shared by every moving object.
RoadUser	Three-lane-per-direction road logic, passing, manual lane changes, emergency yielding.
SidewalkUser	Two bidirectional pedestrian lanes with passing and keep-right head-on avoidance.
BikePathUser	Dedicated two-way bicycle path; normal directional lane plus temporary opposing-lane overtaking.

Model variants use enums - not more subclasses

CarModel

SportsCar
Sedan
Hatchback

MotorcycleModel

SportsBike
Chopper

BusModel

Regular
Articulated

EmergencyVehicleModel

Ambulance
FireTruck
PoliceCar

PedestrianModel

Skinny
Chubby

BicycleModel

Cruiser
BMX

MountainBike

3 TrafficObject: Current Base-Class Brainstorm

These are the properties that currently make sense for every entity. Exact field/property syntax gets finalized in Phase 2.

Candidate member	Reason it belongs in TrafficObject
X, Y	Every entity needs a position on the graphical map.
Direction	Every entity moves left or right on the foundation map.
Lane	Road users, pedestrians, and bicycles all occupy a logical lane/path band.
DesiredSpeed	What the entity wants to achieve.

Candidate member	Reason it belongs in TrafficObject
ActualSpeed	What current traffic/space safely allows.
Width, Height	Different models occupy different amounts of space.
Bounds	Calculated Rectangle used for selection, occupancy checks, passing, lane changes and spawning.

CENTRAL RULE DesiredSpeed = intention. ActualSpeed = what the environment currently permits. This single distinction powers slowing, stopping, passing, queues, manual control, and emergency yielding.

```
Clear path          -> ActualSpeed = DesiredSpeed
Slower entity ahead -> ActualSpeed is reduced
Stopped entity ahead -> ActualSpeed = 0
Safe alternative opens -> change lane/path, then return toward DesiredSpeed
```

Likely universal methods

- Draw(Graphics g) - each concrete class renders itself differently.
- Move() - move according to direction and ActualSpeed.
- Contains(x, y) or equivalent - support mouse selection.
- Bounds as a calculated property - do not duplicate stored geometry if X/Y/Width/Height already define it.

PHASE 2 TODO Decide which methods are abstract, virtual, or inherited unchanged; decide which helpers belong in TrafficObject versus the environment-specific derived classes.

4 Manual Creation & Graphical Placement

Every entity is created by the user. There is no background spawning system.

```
Choose entity type/model/speed in the Add panel
        |
Press Add -> placement mode
        |
Click a valid graphical lane/path/entrance
        |
Program infers lane + direction + off-screen spawn coordinates
        |
Check per-lane cap + free spawn area
        |
Create object just outside the visible map -> smooth entry
```

Entity	What the user clicks	How direction is decided
RoadUser	A specific road lane	The clicked road lane already has a fixed travel direction.
Bicycle	A specific bike-path lane	The bike lane already has a fixed travel direction.
Pedestrian	Left or right entrance of a walking lane	Left entrance -> walks right. Right entrance -> walks left.

Road capacity

3 lanes each direction
Initial hard cap: about 6 RoadUsers per lane
Still reject creation if spawn area is occupied

Pedestrian capacity

2 bidirectional walking lanes
Initial hard cap: about 10 per lane
Spawn area must still be free

Bike capacity

Two-way path: one directional lane each way
Initial hard cap: about 10 per lane
Spawn area must still be free

Smooth exits

Entity moves fully beyond visible edge
Small off-screen buffer continues motion
Then remove it from TrafficObjectList

PLACEMENT UX The instructions panel changes with the selected entity. Invalid clicks or blocked entrances produce a clear message rather than silently stacking objects.

5 RoadUser Behavior

The foundation map uses two opposite carriageways, each with three lanes. No intersection/traffic-light logic is needed.

```
PAVEMENT
BIKE PATH
```

```
<- Lane 3
<- Lane 2
<- Lane 1
========== MEDIAN ==========
-> Lane 1
-> Lane 2
-> Lane 3

BIKE PATH
PAVEMENT
```

Situation	Automatic RoadUser behavior
Path clear	ActualSpeed follows DesiredSpeed.
Slower vehicle ahead	Reduce ActualSpeed enough to follow safely; try a safe adjacent lane.
Stopped vehicle ahead	ActualSpeed becomes 0 if no safe adjacent lane exists.
Adjacent lane becomes free	Normal per-tick logic notices immediately and can pass; no separate retry counter.
Target movement would overlap	Movement/lane change is rejected. Entities never overlap.

NO RETRY TIMER The global Timer already re-evaluates DesiredSpeed vs ActualSpeed every tick. If a lane is blocked now, the entity simply checks again naturally on the next tick.

Reusable occupancy principle

- Before moving forward: is the next Bounds area free?
- Before changing lanes: is the target lane area free?
- Before spawning: is the entrance area free?
- This is the same underlying rule in all three cases.

6 Manual Control Mode

Every entity can be taken over by the player. Movement keys stay universal.

Input	Meaning
W / Up	Increase DesiredSpeed by 1, up to the entity/category maximum.
S / Down	Decrease DesiredSpeed by 1. Dropping below 1 results in 0.
Space	Immediate brake: DesiredSpeed = 0 and ActualSpeed = 0.
A / Left	Request a valid lateral/lane movement to the left.
D / Right	Request a valid lateral/lane movement to the right.

Input	Meaning
H	Context action: horn, siren, bike horn, or pedestrian shout.

MANUAL VS AI The game may reduce ActualSpeed to prevent overlap, but it does NOT automatically pass for the player. The player chooses A/D. W/S only change DesiredSpeed.

Manual context	H action	Looping movement sound
Normal RoadUser	Horn	Engine sound based on simple speed bands.
EmergencyVehicle	Toggle siren (instead of horn)	Engine sound + simple siren WAV behavior.
Bicycle	Bike horn	Pedalling / chain WAV.
Pedestrian	Funny shout / vocal clip	Footstep WAV.

AUDIO LIMIT Use only System.Media.SoundPlayer with simple WAV files. No pitch/RPM simulation, spatial audio, advanced mixing, or external audio libraries.

Focus-mode UI

- When manual mode starts, hide every normal UI panel/control.
- Leave only a small context-sensitive instruction overlay.
- Use different instruction titles/text for Drive Mode, Emergency Drive Mode, Cycle Mode, and Walk Mode.
- Exit manual mode via Escape, any mouse button on the simulation area, or the controlled entity leaving map bounds.
- After exit, restore normal UI. If the entity left the map, it continues through the off-screen buffer and is then deleted.

7 Emergency-Vehicle Siren Logic

Same road rules, but siren ON changes which vehicle is asked to move.

```
NORMAL VEHICLE BLOCKED
Our vehicle cannot reach DesiredSpeed
-> our vehicle tries a safe adjacent lane
-> if none is free, our ActualSpeed is reduced

EMERGENCY + SIREN ON
Emergency vehicle cannot reach DesiredSpeed
-> blocking vehicle tries a safe adjacent lane instead
-> if it cannot move, emergency vehicle slows normally
```

> **WHY THIS IS CLEAN** The project reuses the same lane-change and occupancy function. The only difference is which object receives the lane-change request.

- Siren OFF: emergency vehicle behaves like an ordinary RoadUser.
- Siren ON: car ahead tries to clear the lane if a safe adjacent space exists.
- If no safe move exists, nobody clips or teleports; the emergency vehicle simply slows.
- The same rule naturally re-evaluates on the next global Timer tick.

8 Bicycle Behavior

Simpler than cars, because the path itself has less complexity.

```
BIKE PATH
<- <- <- <-   directional lane
-> -> -> ->   directional lane
```

- Bicycles normally stay in their proper directional lane.
- They still use DesiredSpeed / ActualSpeed and no-overlap Bounds checks.
- If a slower bicycle is ahead, the faster bicycle may temporarily enter the opposing lane to pass.
- A pass starts only if the opposing space is clear enough and no oncoming bicycle is too close.
- Keep-right takes priority: if an oncoming bicycle appears, the passing bicycle returns to its correct lane when safe.
- If it cannot safely return yet, it slows/stops rather than overlapping another model.
- Manual A/D uses the same valid-lane logic; H uses the bicycle horn.

9 Pedestrian Behavior

Two bidirectional walking lanes with a simple keep-right conflict rule.

```
PAVEMENT
Lane 1  <->
Lane 2  <->
```

Situation	Pedestrian behavior
Clear path	ActualSpeed follows DesiredSpeed.
Slower pedestrian ahead	Try the other walking lane; otherwise match/reduce speed.
Head-on pedestrian	Apply keep-right. Each pedestrian tries to move to its own right-hand side.
Target lane occupied	Slow/stop and let the same logic re-evaluate next Timer tick.
Manual control	Same W/S/A/D/Space inputs; H plays a shout/vocal WAV.

NO PREFERRED-LANE PROPERTY Pedestrians do not carry a permanent preferred lane. Same-direction passing and head-on keep-right are resolved only when a conflict actually exists.

10 TrafficObjectList - Phase 3 Design Target

The collection is the heart of the P9 object system and should stay close to Vladimir's course pattern.

```
TrafficObjectList
    SortedList objects

    Count / NextIndex
    indexer [i]
    Add / Remove
    DrawAll(Graphics g)
    CountInLane(...)
    IsAreaFree(...)
    FindRelevantObjectAhead(...)   [if we decide this belongs here]
```

POLYMORPHISM TrafficObjectList stores TrafficObject references. One collection can therefore hold Car, Bus, Motorcycle, EmergencyVehicle, Pedestrian, and Bicycle objects, while Draw/Move still dispatch to the real runtime type.

Phase 3 decisions still to finalize

- Exact SortedList key/index scheme and course-style indexer behavior.
- Which helper searches belong inside TrafficObjectList versus RoadUser/SidewalkUser/BikePathUser.
- How per-lane counts distinguish road direction, bike direction, and pedestrian lanes.
- How occupancy checks efficiently ignore irrelevant entity groups.

11 Serialization & Real Files - Phase 4

Save/Load is mandatory and should be a visible, testable part of the project - not an afterthought.

```
SAVE
SaveFileDialog
```

```
-> FileStream(FileMode.Create)
-> BinaryFormatter / course-style serialization
-> serialize TrafficObjectList (or final chosen root object)
-> actual project file created

LOAD
OpenFileDialog
-> FileStream(FileMode.Open)
-> Deserialize
-> restore concrete classes + state
-> replace current object system
-> Invalidate() / refresh UI
```

State that should survive Save/Load	Notes
Concrete runtime type	A SportsCar still loads as a Car object with SportsCar model; not generic TrafficObject.
Model enum	Car/bus/bike/ped/emergency variant is restored.
X / Y, lane, direction	Loaded scene appears in the same place/state.
DesiredSpeed / ActualSpeed	Traffic state is preserved.
Emergency SirenOn	Object property may be restored; actual sound playback is runtime/UI behavior.

- Mark relevant model/collection classes [Serializable].
- Do NOT serialize Form, Button, PictureBox, Timer, Graphics, SoundPlayer, or other UI/runtime objects.
- Road/background geometry is fixed presentation and does not need to be serialized.
- On load, return to normal UI mode and redraw the restored object system.
- Choose a simple custom extension later if desired; the important part is that Save actually creates a file and Open actually restores it.

> **DEMO GOAL** Create a mixed scene, modify several models/speeds/lanes, save it, reset/close, load it, and visibly restore the same object system.

12 Current Build Checklist

Phase 1 behavior design is essentially done. Today ends when Phase 4 is fully designed.

- [x] Project theme, scope and P9 requirement mapping
- [x] TrafficObject hierarchy and entity categories
- [x] Manual-only placement philosophy
- [x] Road / bike / pavement layouts
- [x] DesiredSpeed / ActualSpeed rule
- [x] Vehicle lane-management and no-overlap rules
- [x] Emergency siren-yield rule
- [x] Bicycle passing + keep-right
- [x] Pedestrian passing + keep-right
- [x] Manual mode UX and control scheme
- [x] Simple SoundPlayer audio concept
- [ ] PHASE 2: finalize exact class properties/methods and abstract/virtual/override ownership
- [ ] PHASE 3: finalize TrafficObjectList and helper responsibilities
- [ ] PHASE 4: finalize serialization root, file flow and restored state
- [ ] STOP FOR TODAY once Phase 4 is finished
- [ ] PHASE 5 tomorrow: WinForms UI layout
- [ ] PHASE 6 tomorrow: mandatory P9 implementation
- [ ] PHASE 7 tomorrow: interactive movement/audio extras
- [ ] PHASE 8 tomorrow: testing, course-scope review, final submission check

13 Design Principles to Protect

These are the rules that have consistently improved the project and should stay visible while coding.

Reuse one rule

Model the underlying principle once
Avoid scenario-specific duplicate methods
Example: DesiredSpeed / ActualSpeed solves many traffic cases

No invisible magic

Every important behavior should be explainable
Prefer loops, conditions, properties and events
Avoid clever advanced C# tricks

Validate before acting

Check capacity before spawn
Check Bounds before movement
Check target space before lane/path changes

Course-first architecture

Stay close to P9 examples where practical
Use standard WinForms components
Every line should be defendable in class

ONE-SENTENCE PROJECT MODEL The Sharp Turn is a manually populated top-down traffic sandbox where polymorphic objects move through road, bike, and pedestrian environments using shared

> speed/occupancy principles, can be directly controlled by the user, and can be saved/restored as a serialized object system.

End of current project bible - update this document whenever a major design decision is locked.